Accueil du niveau

MODULE 4 — MÉTHODOLOGIE DE RÉSOLUTION PYTHON

Transformer un besoin métier en script Python lisible et fiable.

Accueil du niveau

OBJECTIF DU MODULE

- Découper un problème en sous-tâches.
- Concevoir un flux de traitement clair.
- Coder avec des fonctions simples et testables.
- Valider les résultats avant livraison.

Accueil du niveau

DÉMARCHE EN 5 ÉTAPES

1. Comprendre les entrées et sorties attendues.
2. Lister les règles métier explicites.
3. Écrire un pseudo-code court.
4. Implémenter par blocs indépendants.
5. Contrôler et documenter le résultat.

Accueil du niveau

EXEMPLE DE PSEUDO-CODE

```
# Entrée: liste de commandes
# Sortie: CA total par canal

charger_les_donnees()
nettoyer_valeurs_invalides()
calculer_ca_par_canal()
afficher_resume()
```

Accueil du niveau

DÉCOUPER EN FONCTIONS COURTES

```
def nettoyer_prix(prix: float) -> float:
    if prix < 0:
        raise ValueError("prix négatif")
    return round(prix, 2)


def total_ca(lignes: list[dict]) -> float:
    return sum(nettoyer_prix(l["prix"]) for l in
lignes)
```

Accueil du niveau

CHOISIR LES BONNES STRUCTURES

- `list` pour des séquences ordonnées.
- `dict` pour associer clé/valeur métier.
- `set` pour dédupliquer rapidement.
- `tuple` pour des couples immuables simples.

Accueil du niveau

CONTRÔLER LES CAS LIMITES

```
def ratio(success: int, total: int) -> float:
    if total == 0:
        return 0.0
    return success / total

print(ratio(10, 0))  # 0.0 au lieu d'une erreur
```

Accueil du niveau

GESTION D'ERREURS LISIBLE

```
try:
    data = charger_csv("ventes.csv")
except FileNotFoundError:
    print("Fichier introuvable")
except ValueError as err:
    print(f"Donnée invalide: {err}")
```

Accueil du niveau

LISIBILITÉ ET NOMMAGE

- Noms explicites: `prix_net`, `date_commande`.
- Fonctions petites: une intention claire.
- Commentaires ciblés uniquement si nécessaire.
- Docstring sur fonctions publiques.

Accueil du niveau

VALIDATION RAPIDE DES RÉSULTATS

```
def test_total_ca():
    lignes = [{"prix": 10.0}, {"prix": 20.0}]
    assert total_ca(lignes) == 30.0


def test_total_ca_refuse_negatif():
    try:
        total_ca([{"prix": -1.0}])
    except ValueError:
        assert True
```

Accueil du niveau

CHECKLIST DE LIVRAISON D'UN SCRIPT

- Entrées/sorties clairement définies.
- Aucun chemin codé en dur dans le script.
- Erreurs principales capturées.
- Exemple d'exécution fourni.

Accueil du niveau

RÉCAPITULATIF

- La méthode précède l'écriture du code.
- Le découpage en fonctions limite les régressions.
- Validation + lisibilité = base d'un code professionnel.

Speaker notes